TGPS Community X

Architecture Document

Project: TGPS Blockchain Community X

Roles: Member & Admin

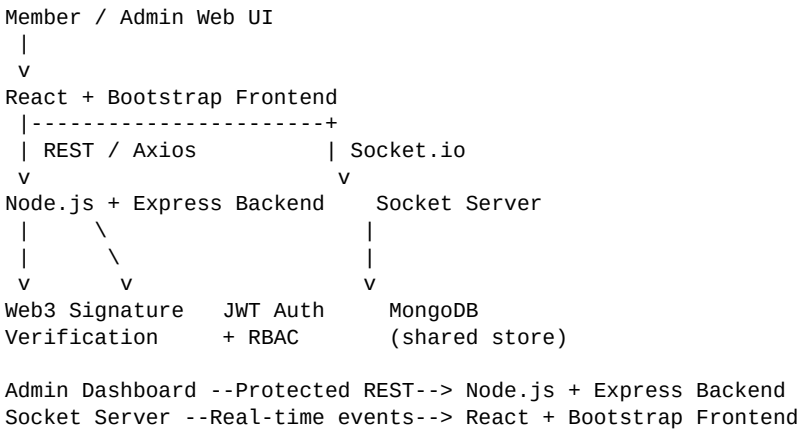
Stack: React + Bootstrap · Node.js/Express · MongoDB/Mongoose · Socket.io · Web3 wallet authentication · JWT

Purpose. This document defines the complete application architecture for a private Community X platform where users authenticate through a blockchain wallet, complete profile and validation steps, enter a member-only community, use real-time group chat, and communicate privately with administrators.

Architecture approach. The design follows a layered documentation pattern — high-level overview, project structure, data models, API contracts, authentication/authorization, service design, real-time processing, frontend architecture, security, testing, observability, deployment, CI/CD, backup/recovery, runbooks, sequence flows, sample code patterns, auditing, the development roadmap, and a day-by-day project timeline.

1. High-level overview

Community X is a Web3-enabled private community application. The frontend provides the user experience, the backend is the source of truth for identity, membership and permissions, MongoDB stores persistent data, Web3 libraries verify wallet ownership, JWT protects application sessions, and Socket.io provides real-time communication.



Core principle: the frontend displays state, but the backend decides whether a wallet is valid, whether a user is a member, what role they have, and whether an action is permitted.

2. Project structure

The backend includes a services layer so controllers stay thin and business rules such as wallet validation, profile validation and conversation permissions live in reusable service classes.

```
/tgps-blockchain-community
|-- client/
|   |-- public/
|   |-- src/
|   |   |-- assets/
|   |   |   |-- icons/
|   |   |   |-- custom-bootstrap.scss
|   |   |   `-- cyberpunk-glow.css
|   |   |-- components/
|   |   |   |-- common/
|   |   |   |-- onboarding/
|   |   |   |-- chat/
|   |   |   `-- admin/
|   |   |-- pages/
|   |   |-- context/
|   |   |-- hooks/
|   |   |-- services/
|   |   |-- utils/
|   |   |-- App.jsx
|   |   `-- main.jsx
|   |-- index.html
|   |-- package.json
|   `-- vite.config.js
|
|-- server/
|   |-- config/
|   |-- controllers/
|   |-- services/
|   |-- middleware/
|   |-- models/
|   |-- routes/
|   |-- sockets/
```

```
|   |-- utils/
|   |-- app.js
|   |-- server.js
|   |-- .env
|   `-- package.json
|
|-- docs/
|-- .gitignore
`-- README.md
```

3. Core concepts and responsibilities

Layer	Primary responsibility
Frontend	UI/UX, routing, forms, wallet connection, chat presentation, client state.
Backend	Authentication, wallet verification, profile validation, membership rules, authorization, APIs.
MongoDB	Users, profiles, wallet identities, group messages, private messages, audit records.
Web3 layer	Wallet connection/signature request and server-side signature verification.
JWT	Authenticated session after successful wallet verification.
Socket.io	Real-time group chat, private chat, online presence and chat events.
Admin layer	Member management, member list, private conversations, moderation/audit actions.

The backend remains the authoritative source for user identity and permissions, keeping a clean separation between frontend UI, backend business rules, database persistence, security and operational concerns.

4. Data models (MongoDB) — detailed

Use Mongoose schemas with explicit field types, indexes and timestamps. Wallet addresses should be normalized before lookup and stored with an index/unique constraint where appropriate.

users

```
{
  _id: ObjectId,
  walletAddress: String,      // unique, indexed
  role: String,                // member | admin
  status: String,              // active | pending | blocked
  profile: {
    fullName: String,
    email: String,
    phone: String,
    city: String,
    bio: String,
    avatar: String
  },
  isWalletVerified: Boolean,
  isProfileComplete: Boolean,
  isCommunityVerified: Boolean,
  lastSeenAt: Date,
  createdAt: Date,
  updatedAt: Date
}
```

groupMessages

```
{
  _id: ObjectId,
  senderId: ObjectId,
```

```
  senderWallet: String,
  message: String,
  messageType: String,   // text | system
  room: String,           // community-x
  editedAt: Date,
  deletedAt: Date,
  createdAt: Date
}
```

privateMessages

```
{
  _id: ObjectId,
  conversationId: String,
  senderId: ObjectId,
  receiverId: ObjectId,
  message: String,
  messageType: String,
  readAt: Date,
  createdAt: Date
}
```

auditLogs

```
{
  _id: ObjectId,
  actorId: ObjectId,
  action: String,
  targetId: ObjectId,
  metadata: Object,
  createdAt: Date
}
```

5. API design & contracts

Base URL: /api/v1. Use a consistent response envelope so the frontend handles success and errors predictably.

```
{
  "status": "success" | "error",
  "data": {},
  "error": null
}
```

Authentication / Web3

Endpoint	Purpose
POST /api/v1/auth/nonce	Generate a short-lived nonce/message for a wallet.
POST /api/v1/auth/verify	Verify the signed message and issue an authenticated session.
GET /api/v1/auth/me	Return the current authenticated user.
POST /api/v1/auth/logout	Invalidate the current session.

User / profile

Endpoint	Purpose
GET /api/v1/users/me	Current profile.
PUT /api/v1/users/profile	Create/update profile.
POST /api/v1/users/validate	Validate profile/community eligibility.
GET /api/v1/users/members	Protected member directory.

Messages

Endpoint	Purpose
GET /api/v1/messages/group	Paginated Community X history.
GET /api/v1/messages/private/:userId	Private conversation history.
Socket.io	Handles live send/receive events.

6. Authentication, authorization & session management

The authentication flow is wallet-based rather than password-based.

```
Wallet
  |
  v
Request nonce
  |
  v
User signs server-provided message
  |
  v
POST signature to backend
  |
  v
Verify signature against wallet address
  |
  v
Find / create user
  |
  v
Issue JWT session
  |
  v
Protected routes / Socket.io authentication
```

- Never accept a wallet address alone as proof of ownership.
- Nonce values should be short-lived and single-use.
- JWT should contain only the identity/authorization claims needed by the application.
- Use HttpOnly Secure cookies where the deployment supports cookie-based sessions; otherwise use a carefully protected token strategy.
- Protect admin endpoints with both authentication and role checks.

Recommended practice: short-lived access tokens, secure cookie handling, refresh/revocation support and fine-grained authorization, adapted here to wallet-based authentication.

7. Business logic & service design

Use thin controllers and service classes for business rules — a controller → service → model layering.

```
services/
|-- AuthService.js
|-- UserService.js
|-- ValidationService.js
|-- GroupChatService.js
|-- PrivateChatService.js
`-- AdminService.js
```

Service	Responsibility
AuthService	Nonce creation, signature verification, user lookup/creation, JWT issuance.
UserService	Profile creation/update and member lookup.

Service	Responsibility
ValidationService	Profile completeness and Community X eligibility rules.
GroupChatService	Message validation, persistence and room rules.
PrivateChatService	Conversation authorization, message persistence and read state.
AdminService	Member administration and audit logging.

8. Real-time communication architecture

Socket.io is used for events that need immediate delivery. REST remains useful for initial history, pagination and operations that do not require a persistent socket.

```
Client                                  Server
 |-- socket connect + JWT ------------->|
 |<---------- authenticated ------------|
 |
 |-- join_room("community-x") --------->|
 |
 |-- send_group_message ---------------->|
 |                                        |-- validate
 |                                        |-- save to MongoDB
 |                                        |-- broadcast
 |<---------- new_group_message ---------|
 |
 |-- private_message -------------------->|
 |                                        |-- authorize pair
 |                                        |-- save
 |                                        |-- emit to receiver
 |<---------- new_private_message --------|
```

- Authenticate the socket connection before joining protected rooms.
- Do not trust senderId supplied by the browser; derive identity from the authenticated socket.
- Validate message length/type on the server.
- Persist messages before or as part of the controlled broadcast flow so chat history remains authoritative.

9. Frontend architecture (React + Bootstrap)

```
src/
|-- components/
|   |-- common/
|   |-- onboarding/
|   |-- chat/
|   `-- admin/
|-- pages/
|   |-- LandingPage.jsx
|   |-- OnboardingPage.jsx
|   |-- CommunityDashboard.jsx
|   `-- AdminDashboard.jsx
|-- context/
|   |-- AuthContext.jsx
|   `-- SocketContext.jsx
|-- hooks/
|-- services/
|-- utils/
`-- App.jsx
```

- AuthContext owns authenticated user, wallet, token/session state and login/logout actions.
- SocketContext owns the Socket.io connection and online/presence state.
- Services isolate Axios/API calls from page components.

- Reusable chat components keep GroupChat and AdminPrivateChat maintainable.
- Use Bootstrap grid/utilities plus project-specific CSS for the cyberpunk/community visual identity.

10. Security measures

- Use HTTPS/TLS in production.
- Use Helmet and secure HTTP headers.
- Rate-limit nonce and authentication endpoints.
- Validate and sanitize profile and message input.
- Prevent NoSQL injection through validation and schema constraints.
- Escape user-generated message content; never render raw HTML from users.
- Enforce wallet address normalization and validation.
- Use RBAC for member/admin access.
- Protect admin actions with audit logs.
- Do not store private keys or seed phrases.
- Use environment variables for JWT secrets, database credentials and Web3 provider configuration.
- Add message/report moderation controls before exposing administrative delete/block actions.

11. Testing strategy

Layer	Coverage
Unit tests	Wallet utilities, nonce generation, validation utilities and services.
Integration tests	Auth/profile/message APIs using Supertest and an isolated test MongoDB.
Socket tests	Connection authentication, room membership, group messages and private messages.
E2E tests	Connect wallet → sign → profile → validation → Community X → send message → admin chat.
Security tests	Invalid signatures, expired nonce, missing JWT, wrong role, unauthorized private conversation and malformed messages.
Dependency checks	npm audit and dependency update review.

12. Observability & monitoring

- Structured backend logs for authentication, validation, chat and admin operations.
- Error aggregation such as Sentry can be added in production.
- Health endpoint: GET /healthz.
- Readiness endpoint: GET /readyz.
- Track MongoDB connection state.
- Track Socket.io connection/disconnection counts.
- Track authentication failures and admin actions.
- Never log private keys, seed phrases, JWT secrets or sensitive personal information.

13. Deployment & scaling

For the first implementation, keep deployment simple. Docker/PM2/Nginx can be introduced after the application is stable.

Development

```
React / Vite -> localhost:5173
Express      -> localhost:5000
MongoDB      -> local MongoDB / Atlas
```

Production

```
Browser
  |
  v
Nginx / CDN
  |
  +--> React static files
  +--> /api + Socket.io
        |
        v
    Node / Express
        |
        v
     MongoDB
```

- Use MongoDB Atlas or a managed MongoDB deployment for production.
- Use HTTPS — wallet providers and secure authentication flows require a secure browser context.
- If traffic grows, horizontally scale API servers and configure Socket.io with a shared adapter such as Redis so users connected to different server instances can receive room events.
- Use PM2 or containers depending on deployment requirements.

14. CI/CD pipeline

```
Pull Request
  |
  v
Lint -> Unit Tests -> Build Frontend -> Backend Tests
  |
  v
Merge to main
  |
  v
Build deployment artifacts
  |
  v
Deploy staging
  |
  v
Smoke tests
  |
  v
Production deployment
  |
  v
Health check
```

- Keep .env files and secrets out of Git.
- Run frontend and backend builds in CI.
- Run API/auth/security tests before deployment.
- Run a basic smoke test after deployment.

- Deploy Socket.io and API changes together when their event contracts change.

15. Backup & disaster recovery

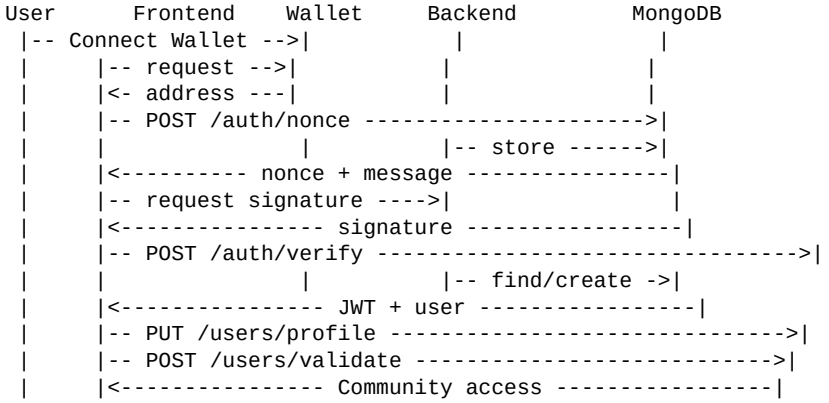
- Enable regular MongoDB backups/snapshots.
- Test restoring backups rather than assuming backups are usable.
- Keep deployment configuration and environment documentation separate from source secrets.
- Maintain a rollback version for the frontend and backend.
- Retain audit logs according to the project's operational policy.

16. Runbook & operational playbooks

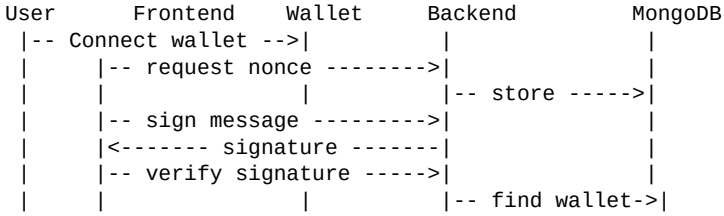
Scenario	Response
Wallet authentication failure	Inspect nonce expiry, signature format, chain/provider configuration and backend verification logs.
MongoDB unavailable	Verify connection string, database status and server health.
Socket.io unavailable	Verify CORS, WebSocket transport, reverse proxy configuration and server process.
Unauthorized admin access attempt	Verify JWT, role lookup and admin middleware.
Message duplication	Inspect client retry behavior and add message IDs/idempotency where needed.
Abusive content	Admin reviews report, applies moderation action; action is written to audit logs.

17. Sequence diagrams — important flows

1) New member onboarding



2) Existing member login



```
 |     |               |          |<- existing ----|
 |     |<------- JWT + member -----|               |
 |     |-- validate access ------->|               |
 |     |<------- community permitted-|             |
```

3) Group chat message

```
Member     React      Socket.io      MongoDB
 |-- type message -->|              |
 |     |-- send_group_message ----->|
 |     |              |-- verify JWT|
 |     |              |-- validate room
 |     |              |-- save message -->|
 |     |<-- new_group_message ------------|
 |<-- render message -|              |
```

4) Private admin chat

```
Member    Frontend    Socket.io     Admin     MongoDB
 |-- open Admin Chat -->|            |          |
 |     |-- join private conversation ->|        |
 |     |              |-- verify relationship   |
 |-- send_private_message ------------>|         |
 |     |              |-- save ---------------->|
 |     |              |-- emit -->  |            |
 |     |              |<-- reply ---|            |
 |     |              |-- save ---------------->|
 |<-- new_private_message --------------|        |
```

18. Sample code patterns

Authentication middleware — conceptual pattern

```
async function authMiddleware(req, res, next) {
  try {
    const token = getTokenFromRequest(req);
    if (!token) {
      return res.status(401).json({ status: "error", error: "Unauthenticated" });
    }
    const payload = verifyJwt(token);
    req.user = payload;
    next();
  } catch (error) {
    next(error);
  }
}
```

Admin middleware — conceptual pattern

```
async function adminMiddleware(req, res, next) {
  if (req.user?.role !== "admin") {
    return res.status(403).json({ status: "error", error: "Forbidden" });
  }
  next();
}
```

Wallet verification service — conceptual pattern

```
class AuthService {
  async verifyWallet({ walletAddress, signature, nonce }) {
    const message = buildLoginMessage(walletAddress, nonce);
    const recoveredAddress = recoverAddress(message, signature);
    if (!sameAddress(recoveredAddress, walletAddress)) {
      throw new Error("Invalid wallet signature");
    }
    const user = await this.findOrCreateUser(walletAddress);
    return this.issueSession(user);
  }
}
```

These snippets are intentionally conceptual; exact Web3 library calls depend on the wallet/provider library selected for the implementation.

19. Auditing & compliance

- Record important admin actions in audit_logs with actor, action, target and timestamp.
- Record membership/validation status changes.
- Record moderation actions such as message deletion or member blocking.
- Do not store private keys or seed phrases.
- Define a data retention policy for profile and chat data.
- Protect sensitive personal profile information and restrict it to authorized use.
- Document who can access member information and administrative chat data.

20. Recommended development roadmap

Phase	Build	Main outcome
1	React UI	Landing, onboarding, community and admin screens
2	Routing	Public/protected/admin route structure
3	Express + MongoDB	User/profile APIs and database
4	Web3 authentication	Wallet → nonce → signature → verification
5	JWT + RBAC	Protected member/admin access
6	Validation flow	New vs existing user and Community X eligibility
7	Socket.io	Real-time group chat
8	Private chat	Member ↔ Admin 1:1 conversations
9	Admin tools	Member list, moderation and audit logs
10	Testing/security	API, WebSocket, E2E and security coverage
11	Deployment	Production build, HTTPS, MongoDB and monitoring

21. Project timeline

Phase	Deliverable	Working days	Start	End
1	React UI — landing, onboarding, community & admin screens	4	Wed, 23 Sep 2026	Sat, 26 Sep 2026
2	Routing — public / protected / admin structure	2	Mon, 28 Sep 2026	Tue, 29 Sep 2026
3	Express + MongoDB — user/profile APIs & database	3	Wed, 30 Sep 2026	Fri, 02 Oct 2026
4	Web3 authentication — wallet → nonce → signature → verify	3	Sat, 03 Oct 2026	Tue, 06 Oct 2026

Phase	Deliverable	Working days	Start	End
5	JWT + RBAC — protected member/admin access	2	Wed, 07 Oct 2026	Thu, 08 Oct 2026
6	Validation flow — new vs existing user eligibility	2	Fri, 09 Oct 2026	Sat, 10 Oct 2026
7	Socket.io — real-time group chat	3	Mon, 12 Oct 2026	Wed, 14 Oct 2026
8	Private chat — member ↔ admin 1:1 conversations	3	Thu, 15 Oct 2026	Sat, 17 Oct 2026
9	Admin tools — member list, moderation, audit logs	3	Mon, 19 Oct 2026	Wed, 21 Oct 2026
10	Testing & security — API, WebSocket, E2E coverage	3	Thu, 22 Oct 2026	Sat, 24 Oct 2026
11	Deployment — production build, HTTPS, monitoring	2	Mon, 26 Oct 2026	Tue, 27 Oct 2026

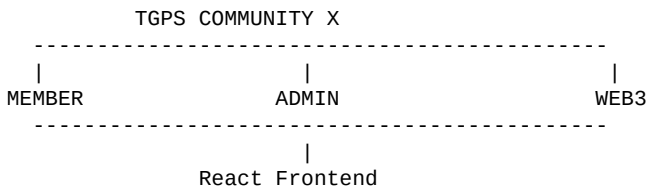
Week-by-week working calendar

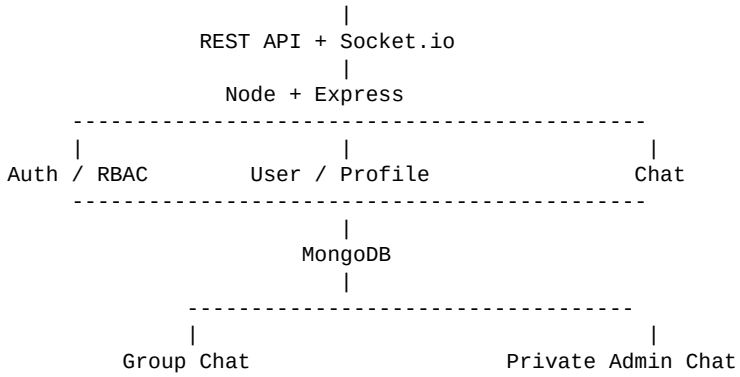
Week	Mon	Tue	Wed	Thu	Fri	Sat	Sun
1	—	—	P1 23 Sep	P1 24 Sep	P1 25 Sep	P1 26 Sep	OFF
2	P2 28 Sep	P2 29 Sep	P3 30 Sep	P3 01 Oct	P3 02 Oct	P4 03 Oct	OFF
3	P4 05 Oct	P4 06 Oct	P5 07 Oct	P5 08 Oct	P6 09 Oct	P6 10 Oct	OFF
4	P7 12 Oct	P7 13 Oct	P7 14 Oct	P8 15 Oct	P8 16 Oct	P8 17 Oct	OFF
5	P9 19 Oct	P9 20 Oct	P9 21 Oct	P10 22 Oct	P10 23 Oct	P10 24 Oct	OFF
6	P11 26 Oct	P11 27 Oct	—	—	—	—	—

Start date	Wed, 23 Sep 2026
Target completion	Tue, 27 Oct 2026
Total working days	30 days

.

22. Final architecture summary





This architecture is based on a layered structure describing project structure, data models, APIs, authentication, service design, frontend architecture, security, testing, monitoring, deployment, CI/CD, recovery, sequence flows, and a concrete project delivery timeline that excludes Sundays.